Web Scraping and Data Frame Creation Code

March 15, 2026

1 Import Libraries and Webpages

[ ]:
```
import requests
import lxml.html as lx
import re
import pandas as pd
import time
```

[ ]:
```
# links to webpages to scrape

dog_url = 'https://dog.rescueme.org/California'
cat_url = 'https://cat.rescueme.org/California'
headers = {
    'User-Agent': "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:144.0)␣
 ↪Gecko/20100101 Firefox/144.0"
}
```

2 Web Scraping Code

2.1 URL Extraction Function

[ ]:
```
def get_pets_from_page(url):
    '''
    A function that extracts the links to all pet profiles located on a webpage.

    Arguments:
    url: URL of a web page

    Return:
    links: A list of all pet profile links found on the webpage
    '''
    # return None of the page doesn't exist
    try:
        result = requests.get(url, headers=headers)
        result.raise_for_status()
    except requests.exceptions.HTTPError:
        return None
```

```
    html = lx.fromstring(result.content)
    pet_links = html.xpath('//div[contains(@class, "card _cl _fa")]//a/@href')

    # return None if empty
    if not pet_links:
        return None

    links = []

    for href in pet_links:
        href = href.strip()
        if not href or href == "#":
            continue
        links.append(href)

    return links
```

2.2 Getter Functions

[ ]:
```
def get_name(html):
    '''
    A function that extracts the name of a pet.

    Arguments:
    html: Link to a pet profile

    Return:
    Name of the pet or None
    '''
    name = html.xpath('//span[contains(@class, "card-pet-name")]/text()')
    if name and name[0]:
        return name[0]
    return None

def get_breed(html):
    '''
    A function that extracts the breed of a pet.

    Arguments:
    html: Link to a pet profile

    Return:
    Breed of the pet or "Unknown"
    '''
    breed = html.xpath('//div[contains(@class, "summary-detail")]//
↪span[contains(@class, "summary-heading")]/text()')
```

```
    if breed and breed[0]:
        return breed[0]
    return "Unknown"

def get_sex(html):
    '''
    A function that extracts the sex of a pet.

    Arguments:
    html: Link to a pet profile

    Return:
    Sex of the pet or "Unknown"
    '''
    sex = html.xpath('//div[contains(@class, "summary-detail")]//
 ↪span[contains(@class, "summary-heading") and contains(text(), "Sex")]')
    if sex and sex[0].tail:
        return sex[0].tail.strip()
    return "Unknown"

def get_age(html):
    '''
    A function that extracts the age of a pet.

    Arguments:
    html: Link to a pet profile

    Return:
    Age of the pet or "Unknown"
    '''
    age = html.xpath('//div[contains(@class, "summary-detail")]//
 ↪span[contains(@class, "summary-heading") and contains(text(), "Age")]')
    if age and age[0].tail:
        return age[0].tail.strip()
    return "Unknown"

def get_county(html):
    '''
    A function that extracts the county a pet is located in.

    Arguments:
    html: Link to a pet profile

    Return:
    County of the pet or None
    '''
    location = html.xpath('//div[contains(@class,"contact-content")]/p')
```

```
    if not location:
        return None
    lines = location[0].xpath('.//text()')
    lines = [line.strip() for line in lines if line.strip()]
    for line in lines:
        if "county" in line.lower():
            return line
    return None

def get_urgent(html):
    '''
    A function that extracts the urgent status of a pet.

    Arguments:
    html: Link to a pet profile

    Return:
    "Yes" or "No"
    '''
    urgent = html.xpath('//div[contains(@class,"card-urgent")]/text()')
    name = get_name(html)
    if name is not None:
        name = name.split()
    else:
        name = []
    if urgent or any("urgent" in w.lower() for w in name):
        return "Yes"
    else:
        return "No"

def get_dog_compatibility(html):
    '''
    A function that extracts the level of compatibility a pet has with dogs.

    Arguments:
    html: Link to a pet profile

    Return:
    "Good", "Not Good", or "Unknown"
    '''
    compatibility = html.xpath('//h3[contains(text(), "Compatibility")]/
 ↪following-sibling::ul[contains(@class,"description-list")]/li/text()')
    compatibility = [i.strip().lower() for i in compatibility if i.strip()]
    for item in compatibility:
        if "dog" in item:
            if "good" in item:
                return "Good"
```

```
            if "not good" in item:
                return "Not Good"
    return "Unknown"

def get_cat_compatibility(html):
    '''
    A function that extracts the level of compatibility a pet has with cats.

    Arguments:
    html: Link to a pet profile

    Return:
    "Good", "Not Good", or "Unknown"
    '''
    compatibility = html.xpath('//h3[contains(text(), "Compatibility")]/
 ↪following-sibling::ul[contains(@class,"description-list")]/li/text()')
    compatibility = [i.strip().lower() for i in compatibility if i.strip()]
    for item in compatibility:
        if "cat" in item:
            if "good" in item:
                return "Good"
            if "not good" in item:
                return "Not Good"
    return "Unknown"

def get_kid_compatibility(html):
    '''
    A function that extracts the level of compatibility a pet has with kids.

    Arguments:
    html: Link to a pet profile

    Return:
    "Good", "Not Good", or "Unknown"
    '''
    compatibility = html.xpath('//h3[contains(text(), "Compatibility")]/
 ↪following-sibling::ul[contains(@class,"description-list")]/li/text()')
    compatibility = [i.strip().lower() for i in compatibility if i.strip()]
    for item in compatibility:
        if "not kid" in item:
                return "Not Good"
        if "kid" in item:
            if "good" in item:
                return "Good"
            if "not good" in item:
                return "Not Good"
    return "Unknown"
```

```
def get_energy_personality(html):
    '''
    A function that extracts the level of energy of a pet.

    Arguments:
    html: Link to a pet profile

    Return:
    "Low", "Average", "High", or "Unknown"
    '''
    personality = html.xpath('//h3[contains(text(), "Personality")]/
 ↪following-sibling::ul[contains(@class,"description-list")]/li/text()')
    personality = [i.strip().lower() for i in personality if i.strip()]
    for item in personality:
        if "energy" in item:
            if "average" in item:
                return "Average"
            if "low" in item:
                return "Low"
            if "high" in item:
                return "High"
    return "Unknown"

def get_temperament_personality(html):
    '''
    A function that extracts the temperament type of a pet.

    Arguments:
    html: Link to a pet profile

    Return:
    "Submissive", "Average", "Dominant", or "Unknown"
    '''
    personality = html.xpath('//h3[contains(text(), "Personality")]/
 ↪following-sibling::ul[contains(@class,"description-list")]/li/text()')
    personality = [i.strip().lower() for i in personality if i.strip()]
    for item in personality:
        if "temperament" in item:
            if "average" in item:
                return "Average"
        if "dominant" in item:
            return "Dominant"
        if "submissive" in item:
                return "Submissive"
    return "Unknown"
```

```
def get_fixed_health(html):
    '''
    A function that extracts the sterilization status of a pet.

    Arguments:
    html: Link to a pet profile

    Return:
    "Yes", "No", or "Unknown"
    '''
    health = html.xpath('//h3[contains(text(), "Health")]/following-sibling::
 ↪ul[contains(@class,"description-list")]/li/text()')
    health = [i.strip().lower() for i in health if i.strip()]
    for item in health:
        if "spay" in item or "neuter" in item:
            if "need" in item:
                return "No"
            else:
                return "Yes"
    return "Unknown"

def get_vaccine_health(html):
    '''
    A function that extracts the vaccination status of a pet.

    Arguments:
    html: Link to a pet profile

    Return:
    "Yes", "No", or "Unknown"
    '''
    health = html.xpath('//h3[contains(text(), "Health")]/following-sibling::
 ↪ul[contains(@class,"description-list")]/li/text()')
    health = [i.strip().lower() for i in health if i.strip()]
    for item in health:
        if "vaccin" in item:
            if "need" in item:
                return "No"
            else:
                return "Yes"
    return "Unknown"

def get_description(html):
    '''
    A function that extracts the description of the pet.

    Arguments:
```

```
    html: Link to a pet profile

    Return:
    Text describing the pet or None
    '''
    description = html.xpath('//p[contains(@class, "animal-description")]/
↪text()')
    if description and description[0]:
        return description[0]
    return None
```

2.3 Final Web Scraping Function

[ ]:
```
def scrape_pets_from_urls(pet_urls, delay=1.0):
    '''
    A function that extracts the information of all pets on a webpage and␣
↪creates a data frame.

    Arguments:
    pet_urls: List of links to pets' profiles
    delay: Programmed pause between requests

    Return:
    df: Pandas data frame with the information of all pets on a webpage
    '''
    all_pets = []

    for i, url in enumerate(pet_urls, 1):
        print(f"Scraping pet {i}/{len(pet_urls)}: {url}")
        try:
            r = requests.get(url, headers=headers)
            r.raise_for_status()
            html = lx.fromstring(r.text)

            pet_data = {
                "Name": get_name(html),
                "Breed": get_breed(html),
                "Sex": get_sex(html),
                "Age": get_age(html),
                "County": get_county(html),
                "Urgent Status": get_urgent(html),
                "Dog Compatibility": get_dog_compatibility(html),
                "Cat Compatibility": get_cat_compatibility(html),
                "Kid Compatibility": get_kid_compatibility(html),
                "Energy": get_energy_personality(html),
                "Temperament": get_temperament_personality(html),
                "Spayed/Neutered": get_fixed_health(html),
```

```
                "Vaccinated": get_vaccine_health(html),
                "Description": get_description(html),
                "URL": url
            }

            all_pets.append(pet_data)

        except Exception as e:
            print(f"Error scraping {url}: {e}")

        time.sleep(delay)

    df = pd.DataFrame(all_pets)
    return df
```

3 Implementation

[ ]:
```
# scraping the information of all 250 dogs

pet_urls = get_pets_from_page(dog_url)
df = scrape_pets_from_urls(pet_urls, delay=1.0)
```

[ ]:
```
# exporting the data frame for dogs

df.to_csv("dogs_california.csv", index=False)
```

[ ]:
```
# scraping the information of all 250 cats

pet_urls = get_pets_from_page(cat_url)
df = scrape_pets_from_urls(pet_urls, delay=1.0)
```

[ ]:
```
# exporting the data frame for cats

df.to_csv("cats_california.csv", index=False)
```

[ ]:
```
# combining the dogs' and cats' data frames into one

df_dogs = pd.read_csv('dogs_california.csv')
df_cats = pd.read_csv('cats_california.csv')

# create a new column to categorize by species

df_dogs['Species'] = 'Dog'
df_cats['Species'] = 'Cat'

df_combined = pd.concat([df_dogs, df_cats], ignore_index=True)
```

```
cols = df_combined.columns.tolist()
cols.insert(1, cols.pop(cols.index('Species')))
df_combined = df_combined[cols]

print(df_combined.head())
```

[ ]:
```
# exporting the data frame for dogs and cats

df_combined.to_csv("dogs_cats_california.csv", index=False)
```